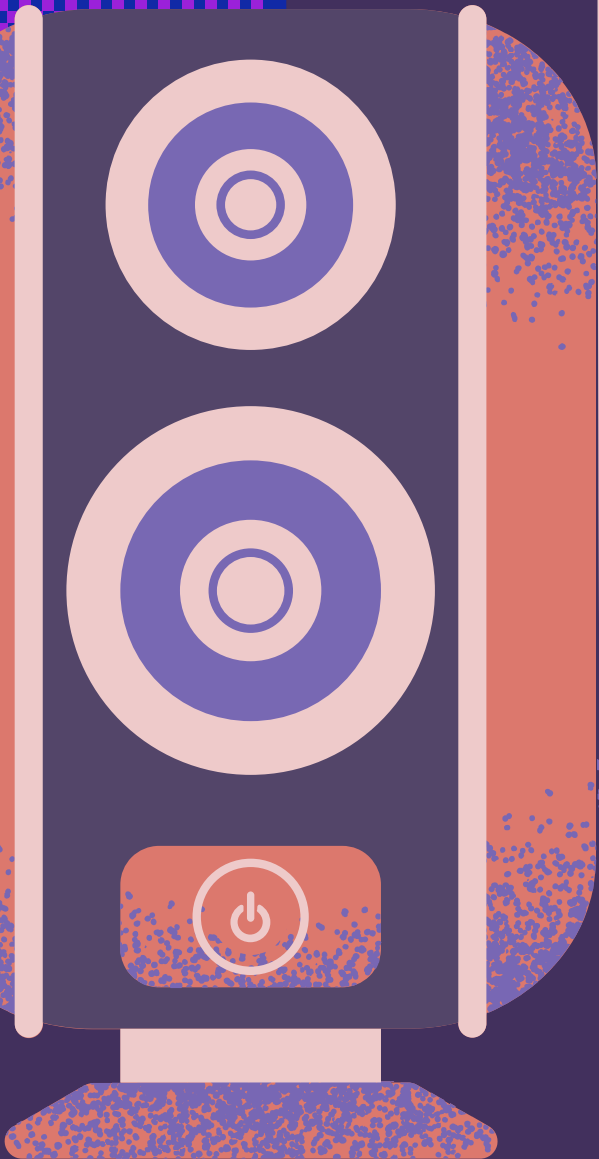


# ASTROCADE



# PLANTEAMIENTO DEL PROBLEMA

¿QUÉ PROBLEMA SE IDENTIFICÓ Y QUÉ  
NECESIDAD SE BUSCA SOLUCIONAR?

Operación Tradicional  
Obsoleta: Los arcades antiguos dependían de fichas físicas o efectivo, dificultando el control de saldos y registros.

Falta de Automatización:  
Ausencia de un sistema centralizado para gestionar perfiles de usuarios de forma independiente y segura.

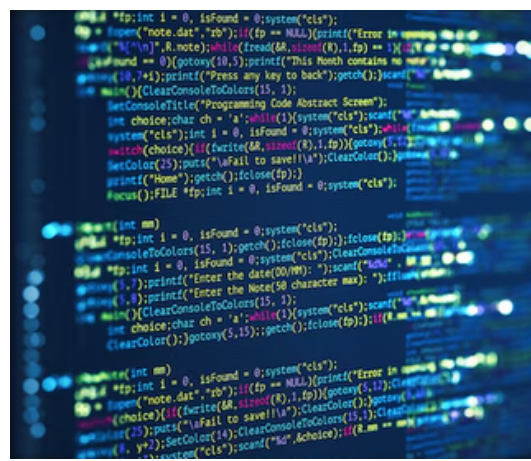
Falta de Automatización:  
Ausencia de un sistema centralizado para gestionar perfiles de usuarios de forma independiente y segura.

Necesidad Identificada: Requerimiento de un software robusto y escalable en Java que automatice el ciclo de vida de un arcade moderno mediante tarjetas electrónicas recargables, garantizando la integridad de datos y control de inventarios.

# ¿QUE ES ASTROCADE?

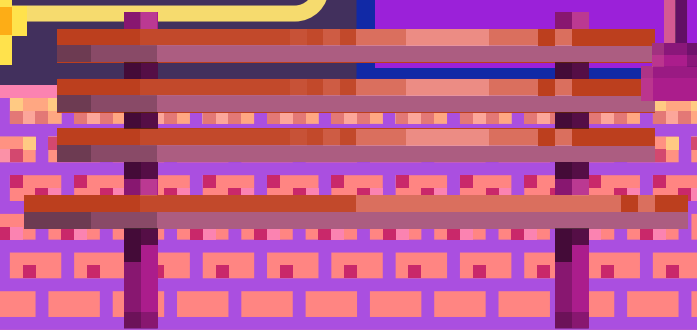
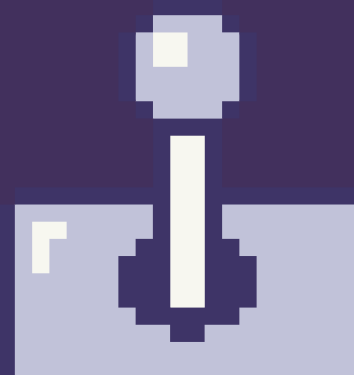
# SIMULACIÓN ESTRUCTURADA

# Modelo desarrollado en Java que replica el funcionamiento operativo de un arcade completo



# ECOSISTEMA COMPLETO

Integra y gestiona  
dinámicamente  
usuarios, tarjetas,  
máquinas de juego y  
centros de redención  
de premios.

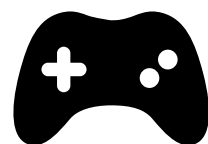


# CLASES PADRE



## TARJETA

Clase base abstracta que custodia el saldo de jugadas y tickets. Sus hijas (Basica, Gold, Diamond) aplican polimorfismo para calcular descuentos personalizados.



## MAQUINAS

Simula las atracciones del arcade. Evalúa estados (Activa/Mantenimiento), descuenta saldos y da paso a máquinas especializadas mediante herencia



## USUARIOS

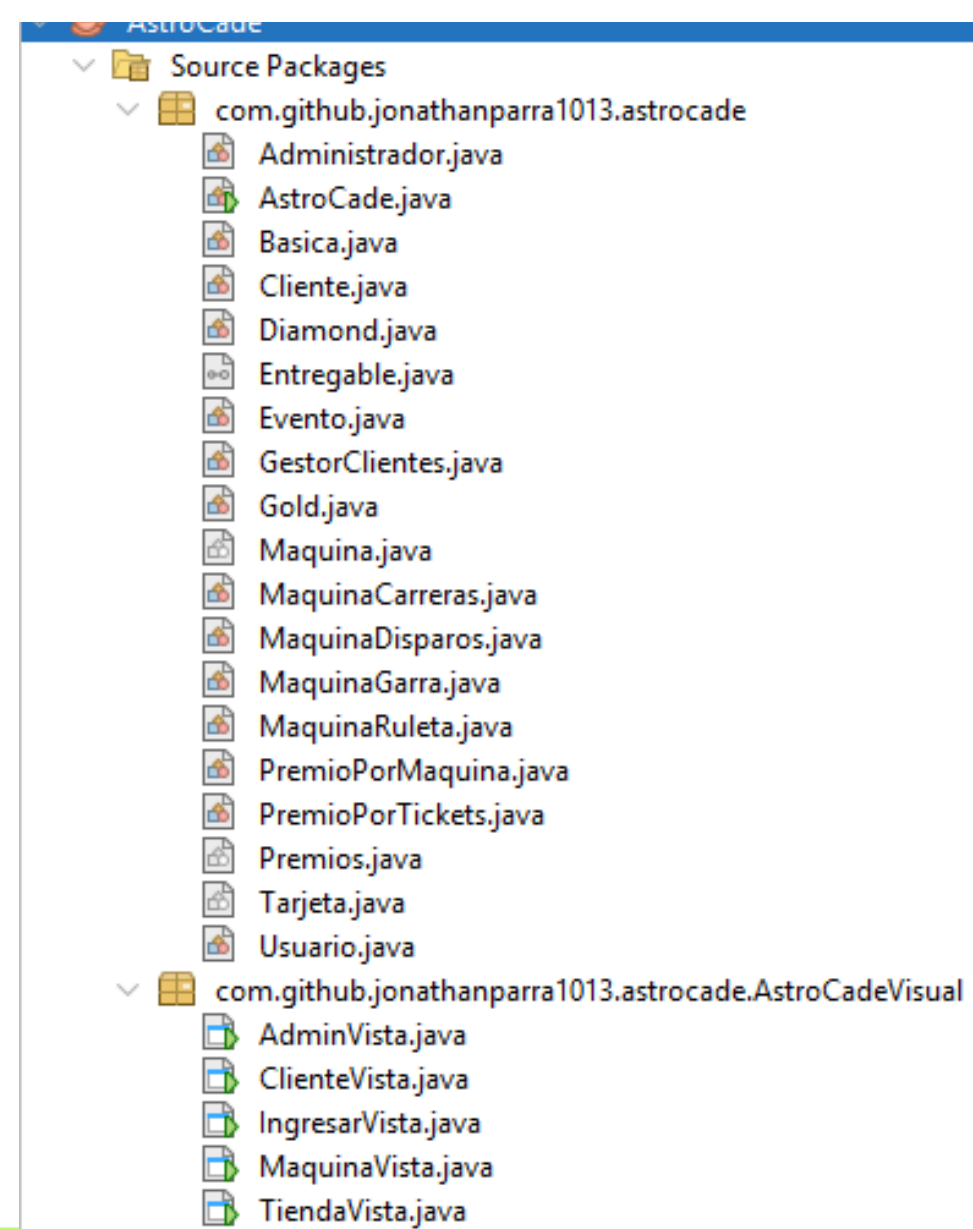
Clase base con datos de identidad. Se especializa en Cliente (con validación automática de cumpleaños) y Administrador (con seguridad por contraseña).



## PREMIOS

Controla el inventario y stock de artículos en tiempo real, validando los tickets acumulados en la tarjeta del cliente para completar el canje.

# PAQUETES



```
com.github.jonathanparra1013.astrocade
├── Administrador.java
├── AstroCade.java
├── Basica.java
├── Cliente.java
├── Diamond.java
├── Entregable.java
├── Evento.java
├── GestorClientes.java
├── Gold.java
├── Maquina.java
├── MaquinaCarreras.java
├── MaquinaDisparos.java
├── MaquinaGarra.java
├── MaquinaRuleta.java
├── PremioPorMaquina.java
├── PremioPorTickets.java
├── Premios.java
├── Tarjeta.java
├── Usuario.java
└── com.github.jonathanparra1013.astrocade.AstroCadeVisual
    ├── AdminVista.java
    ├── ClienteVista.java
    ├── IngresarVista.java
    ├── MaquinaVista.java
    └── TiendaVista.java
```

# CLASE PADRE USUARIO



```
5  package com.github.jonathanparra1013.astrocade;
6  import java.util.Date;
7  import java.util.List;
8  /**
9   *
10   * @author jonap
11   */
12
13
14  ✓ public class Usuario {
15      protected String id;
16      protected String nombre;
17      protected String email;
18
19  ✓  public Usuario(String id, String nombre, String email) {
20      this.id = id;
21      this.nombre = nombre;
22      this.email = email;
23  }
24
25      public boolean login() {
26          System.out.println(nombre + " ha iniciado sesion.");
27          return true;
28      }
29  }
```



# CLASE HIJA CLIENTE



```
14  class Cliente extends Usuario {
15      private Date fechaNacimiento;
16
17      public Cliente(String id, String nombre, String email, Date fechaNacimiento) {
18          super(id, nombre, email);
19          this.fechaNacimiento = fechaNacimiento;
20      }
21
22      public boolean esSuCumpleanosHoy() {
23          if (fechaNacimiento == null) return false;
24          Calendar calNac = Calendar.getInstance();
25          calNac.setTime(fechaNacimiento);
26
27          Calendar calHoy = Calendar.getInstance(); // Fecha actual del sistema
28
29          return (calNac.get(Calendar.MONTH) == calHoy.get(Calendar.MONTH)) &&
30                 (calNac.get(Calendar.DAY_OF_MONTH) == calHoy.get(Calendar.DAY_OF_MONTH));
31      }
32
33      public void verPuntajes(Tarjeta tarjeta) {
34          System.out.println("Saldo de jugadas: " + tarjeta.getSaldoJugadas() + " Tickets: " + tarjeta.get
35      }
36
37      public Date getFechaNacimiento() {
38          return fechaNacimiento;
39      }
40  }
```



# CLASE HIJA ADMIN



```
10  */
11  ✓ class Administrador extends Usuario {
12      private String password;
13
14      public Administrador(String id, String nombre, String email, String password) {
15          super(id, nombre, email);
16          this.password = password;
17      }
18
19      public boolean validarPassword(String passIngresada) {
20          return this.password.equals(passIngresada);
21      }
22
23      public void ponerMaquinaEnMantenimiento(Maquina maquina) {
24          maquina.setEstado("Mantenimiento");
25          System.out.println("Maquina " + maquina.getNombre() + " puesta en mantenimiento.");
26      }
27
28      public void recargarTarjeta(Tarjeta tarjeta, int cantidad) {
29          tarjeta.recargarSaldo(cantidad);
30          System.out.println("Tarjeta recargada con exitos. Nuevo saldo: " + tarjeta.getSaldoJugadas());
31      }
32
33      public void cambiarEstadoMaquina(Maquina maquina, String nuevoEstado) {
34          maquina.setEstado(nuevoEstado);
35          System.out.println("La maquina " + maquina.getNombre() + " ahora esta: " + nuevoEstado);
36      }
37  }
```



# CLASE ABSTRACT TARJETA

```
11  public abstract class Tarjeta {
12      protected String idTarjeta;
13      protected int saldoJugadas;
14      protected int saldoTickets;
15      protected int limiteSaldo;
16
17  public Tarjeta(String idTarjeta, int limiteSaldo) {
18      this.idTarjeta = idTarjeta;
19      this.limiteSaldo = limiteSaldo;
20      this.saldoJugadas = 0;
21      this.saldoTickets = 0;
22  }
23
24  public abstract double calcularDescuento(double costoBase);
25
26  public void recargarSaldo(int cantidad) {
27      if ((this.saldoJugadas + cantidad) <= limiteSaldo) {
28          this.saldoJugadas += cantidad;
29      } else {
30          System.out.println("Error: Supera el límite de saldo de este tipo de tarjeta.");
31      }
32  }
33
34  // Renombrado para que coincida con las máquinas
35  public boolean descontarSaldo(double costo) {
36      if (saldoJugadas >= costo) {
37          saldoJugadas -= costo;
38          return true;
39      }
40      return false;
41  }
42
43  // Renombrado para coincidir con las máquinas
44  public void agregarTickets(int tickets) {
45      this.saldoTickets += tickets;
46  }
```



# CLASES TARJETA



```
2  ✓ class Basica extends Tarjeta {  
3      public Basica(String idTarjeta) {  
4          super(idTarjeta, 50000); // Límite de saldo  
5      }  
6  
7      @Override  
8      public double calcularDescuento(double costoBase) {  
9          return costoBase; // 0% de descuento  
10     }  
11 }
```

```
12  ✓ class Gold extends Tarjeta {  
13      public Gold(String idTarjeta) {  
14          super(idTarjeta, 200000);  
15      }  
16  
17      @Override  
18      public double calcularDescuento(double costoBase) {  
19          return costoBase * 0.90; // 10% de descuento  
20      }  
21 }
```

```
class Diamond extends Tarjeta {  
    public Diamond(String idTarjeta) {  
        super(idTarjeta, 1000000);  
    }  
  
    @Override  
    public double calcularDescuento(double costoBase) {  
        return costoBase * 0.75; // 25% de descuento  
    }  
}
```



# CLASE EVENTO

```
public class Evento {  
    private String nombre;  
    private String tipo; // "BlackFriday", "Cumpleanos", "Normal"  
  
    public Evento(String nombre, String tipo) {  
        this.nombre = nombre;  
        this.tipo = tipo;  
    }  
  
    // Métodos Getter  
    public String getNombre() {  
        return nombre;  
    }  
  
    public String getTipo() {  
        return tipo;  
    }  
    public void setNombre(String nombre) {  
        this.nombre = nombre;  
    }  
    public void setTipo(String tipo) {  
        this.tipo = tipo;  
    }  
}
```



# INTERFAZ ENTREGABLE

```
public interface Entregable {  
  
    String entregarPremio();  
  
}
```



# CLASE PADRE MAQUINA



```
12 public abstract class Maquina {
13
14     private String nombre;
15     private double precioPartida;
16     private String estado; // Nuevo atributo
17
18     public Maquina(String nombre, double precioPartida) {
19         this.nombre = nombre;
20         this.precioPartida = precioPartida;
21         this.estado = "Activa"; // Estado por defecto
22     }
23
24     public abstract void jugar(Tarjeta tarjeta);
25
26     public String getNombre() {
27         return nombre;
28     }
29
30     public double getPrecioPartida() {
31         return precioPartida;
32     }
33
34     // Nuevos métodos para el Administrador
35     public String getEstado() {
36         return estado;
37     }
38
39     public void setEstado(String estado) {
40         this.estado = estado;
41     }
42
43     public void mostrarInformacionMaquina() {
44         System.out.println("Maquina: " + nombre + " | Estado: " + estado);
45         System.out.println("Precio por partida: $" + precioPartida);
46     }
47 }
```



# CLASE HIWA MAQUINA CARRERAS



```
public class MaquinaCarreras extends Maquina {  
  
    public MaquinaCarreras(String nombre, double precioPartida) {  
        super(nombre, precioPartida);  
    }  
  
    @Override  
    public void jugar(Tarjeta tarjeta) {  
  
        if (tarjeta.descontarSaldo(getPrecioPartida())) {  
  
            System.out.println("////////////////////////////////////");  
            System.out.println("        JUEGO DE CARRERAS");  
            System.out.println("////////////////////////////////////");  
            System.out.println("Maquina: " + getNombre());  
  
            int posicion = (int) (Math.random() * 3) + 1;  
  
            System.out.println("Carrera terminada.");  
            System.out.println("Llegaste en la posicion: " + posicion);  
  
            int tickets;  
  
            if (posicion == 1) {  
                tickets = 50;  
            } else if (posicion == 2) {  
                tickets = 20;  
            } else {  
                tickets = 5;  
            }  
  
            tarjeta.agregarTickets(tickets);  
  
            System.out.println("Tickets ganados: " + tickets);  
            System.out.println("Tickets disponibles: " + tarjeta.getTickets());  
            System.out.println("Saldo restante: $" + tarjeta.getSaldo());  
  
        } else {  
            System.out.println("Saldo insuficiente para jugar.");  
        }  
    }  
}
```



# CLASE HIJA MAQUINA RULETA



```
public class MaquinaRuleta extends Maquina {  
  
    private PremioPorMaquina[] premios;  
  
    public MaquinaRuleta(String nombre, double precioPartida,  
        PremioPorMaquina[] premios) {  
  
        super(nombre, precioPartida);  
        this.premios = premios;  
    }  
  
    @Override  
    public void jugar(Tarjeta tarjeta) {  
  
        // Verificar y descontar saldo  
        if (!tarjeta.descontarSaldo(getPrecioPartida())) {  
  
            System.out.println("////////////////////////////////////");  
            System.out.println("                MAQUINA RULETA");  
            System.out.println("////////////////////////////////////");  
            System.out.println("Saldo insuficiente para jugar.");  
  
            return;  
        }  
  
        System.out.println("////////////////////////////////////");  
        System.out.println("                MAQUINA RULETA");  
        System.out.println("////////////////////////////////////");  
  
        System.out.println("Maquina: " + getNombre());  
        System.out.println("Precio: $" + getPrecioPartida());  
  
        PremioPorMaquina premio = buscarPremioDisponible();  
  
        if (premio == null) {  
  
            System.out.println("No hay premios disponibles.");  
            System.out.println("La maquina no puede entregar premios.");  
  
            return;  
        }  
  
        System.out.println("La ruleta esta girando...");  
  
        int numero = (int) (Math.random() * 100) + 1;  
  
        System.out.println("Numero obtenido: " + numero);  
    }  
}
```



# CLASE PADRE PREMIOS



```
public class PremioPorTickets extends Premios {

    private int costoTickets;

    public PremioPorTickets(String nombre, int stock, int costoTickets) {

        super(nombre, stock);

        if (costoTickets > 0) {
            this.costoTickets = costoTickets;
        } else {
            this.costoTickets = 1;
        }
    }

    public int getCostoTickets() {
        return costoTickets;
    }

    @Override
    public void mostrarPremio() {

        System.out.println("//////////PREMIO POR TICKETS//////////");
        System.out.println("Premio: " + getNombre());
        System.out.println("Costo: " + costoTickets + " tickets");
        System.out.println("Stock disponible: " + getStock());
    }

    public void reclamarPremio(Tarjeta tarjeta) {

        if (!hayStock()) {

            System.out.println("//////////PREMIO AGOTADO//////////");
            System.out.println("Premio: " + getNombre());
            System.out.println("Actualmente no hay stock.");

            return;
        }

        if (tarjeta.getTickets() >= costoTickets) {

            tarjeta.descontarTickets(costoTickets);

            reducirStock();
        }
    }
}
```



# CLASE HIJA PREMIO MAQUINA



```
public abstract class PremioPorMaquina extends Premios implements Entregable {

    public PremioPorMaquina(String nombre, int stock) {
        super(nombre, stock);
    }

    @Override
    public void mostrarPremio() {

        System.out.println("//////////PREMIO DE MAQUINA//////////");
        System.out.println("Premio: " + getNombre());
        System.out.println("Stock disponible: " + getStock());
    }

    public String entregarPremio(){

        if (reducirStock()) {

            System.out.println("//////////PREMIO GANADO//////////");
            System.out.println("Premio: " + getNombre());
            System.out.println("¡Felicidades!");
            System.out.println("Stock restante: " + getStock());

        } else {

            System.out.println("//////////PREMIO AGOTADO//////////");
            System.out.println("Premio: " + getNombre());

        }
        return null;
    }
}
```



# INTERFAZ GRAFICA (INGRESAR VISTA)



BIENVENIDO A ASTROCADE  
(Simulacion Arcade)

Seleccione Tipo de Usuario



# INTERFAZ GRAFICA (ADMIN VISTA)



ASTROCADE

Salir

ADMINISTRADOR

Seleccione maquina que desee modificar 

Seleccione

Poner maquina en Mantenimiento: 

Mantenimiento

Recargar Tarjeta a Cliente 

Recargar Tarjeta

CUENTAS

| Nombre | Correo | Tipo |
|--------|--------|------|
|        |        |      |

Registrar Cliente

Id Tarjeta

Nombre

Correo

Saldo

id

nombre

correo

saldo

Registrar



# INTERFAZ GRAFICA (CLIENTE VISTA)



ASTROCADE

Salir

CLIENTE 

Seleccione ▼

| Nombre | Saldo | Tickets | Premios |
|--------|-------|---------|---------|
|        |       |         |         |

Ver puntajes? 

Puntaje

Actualizar Info Cliente

| Saldo de Jugadas | Tickets |
|------------------|---------|
|                  |         |

Verificar Cumpleaños? 

cumple

JUGAR

TIENDA



# INTERFAZ GRAFICA (MAQUINA VISTA)



ASTROCADE

Salir

Seleecone que Maquina va a jugar: Seleecone

Jugar



# INTERFAZ GRAFICA (TIENDA VISTA)



ASTROCADE

Salir

TIENDA DE PREMIOS

| Nombre | Tickets | Saldo |
|--------|---------|-------|
|        |         |       |

Cargar Informacion Cliente

LISTA PREMIOS

| Premio | Valor Tickets |
|--------|---------------|
|        |               |

Selecciona Premio

Selecione ▼

REDIMIR



# LISTAS



```
public static void agregarCliente(Cliente cliente, Tarjeta tarjeta) {  
    listaGlobal.add(new ClienteConTarjeta(cliente, tarjeta));  
}
```

```
public class Cliente extends Usuario {  
    private Date fechaNacimiento;  
    private List<String> premiosRedimidos = new ArrayList<>();  
}
```

```
import java.util.ArrayList;  
import java.util.Date;  
import java.util.List;  
  
public class GestorClientes {
```





GRACIAS POR  
JUGAR!!